Practical SQL Lab – Simplified Star Schema

Scenario:

You are working as a database analyst for a small retail store. The store tracks products, customers, and sales. Your task is to create tables, insert data, and run queries to analyze sales.

There are three tables:

Dim_Product: Product_ID, Product_Name, Category, Price

Dim_Customer: Customer_ID, Customer_Name, Loyalty_Status

Fact_Sales: Sales_ID, Product_ID, Customer_ID, Sales_Amount

You are provided with the sample data to insert:

Dim_Product:

(101, 'Laptop', 'Electronics', 1000), (102, 'Desk', 'Furniture', 700)

Dim_Customer:

(1, 'Alice', 'Premium'), (2, 'Bob', 'Regular')

Fact_Sales:

(1, 101, 1, 1000), (2, 102, 2, 700)

Tasks\ Questions:

Q1 – CREATE Tables with primary keys and foreign keys.

Q2 – INSERT Data, Insert exactly the rows above using the values in parentheses.

Q3 – SELECT Queries.

Select all products in category 'Electronics' or 'Furniture' with price > 500,

Order by price descending and show only top 2.

Q4 – UPDATE & DELETE

Update the price of 'Laptop' to 1200.

Delete the product 'Desk'.

Q5 – Aggregate Functions

Calculate total sales (**SUM**)

Calculate average sale per transaction (**AVG**)

Count number of sales transactions (**COUNT**)

Find maximum and minimum sale (**MAX, MIN**)

Q6 – GROUP BY & HAVING

Show total sales per product

Show products with total sales > 1000

Practical SQL Lab – Student Answers

Q1 – CREATE Tables

```sql
-- Create Product table
CREATE TABLE Dim_Product (
    Product_ID INT PRIMARY KEY,
    Product_Name VARCHAR(50),
    Category VARCHAR(50),
    Price DECIMAL(10,2)
);

-- Create Customer table
CREATE TABLE Dim_Customer (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(50),
    Loyalty_Status VARCHAR(20)
);

-- Create Fact Sales table
CREATE TABLE Fact_Sales (
    Sales_ID INT PRIMARY KEY,
    Product_ID INT,
    Customer_ID INT,
    Sales_Amount DECIMAL(10,2),
    FOREIGN KEY (Product_ID) REFERENCES Dim_Product(Product_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Dim_Customer(Customer_ID)
);
```

Q2 – INSERT Data

```sql
-- Insert products
INSERT INTO Dim_Product VALUES
(101, 'Laptop', 'Electronics', 1000),
(102, 'Desk', 'Furniture', 700);

-- Insert customers
INSERT INTO Dim_Customer VALUES
(1, 'Alice', 'Premium'),
(2, 'Bob', 'Regular');

-- Insert sales
INSERT INTO Fact_Sales VALUES
(1, 101, 1, 1000),
(2, 102, 2, 700);
```

Q3 – SELECT Queries

```
SELECT Product_Name, Category, Price
FROM Dim_Product
WHERE Category IN ('Electronics','Furniture') AND Price > 500
ORDER BY Price DESC
LIMIT 2;
```

Expected Result:

Product_Name	Category	Price
Laptop	Electronics	1000
Desk	Furniture	700

Q4 – UPDATE & DELETE

```
-- Update price of Laptop

UPDATE Dim_Product
SET Price = 1200
WHERE Product_Name = 'Laptop';

-- Delete Desk product
DELETE FROM Dim_Product
WHERE Product_Name = 'Desk';
```

Expected Changes:

- `Laptop` price is now 1200
- `Desk` is removed from the Dim_Product table

Q5 – Aggregate Functions

```
-- Total sales
SELECT SUM(Sales_Amount) AS Total_Sales FROM Fact_Sales;

-- Average sale per transaction
SELECT AVG(Sales_Amount) AS Avg_Sale FROM Fact_Sales;

-- Count number of transactions
SELECT COUNT(Sales_ID) AS Total_Transactions FROM Fact_Sales;

-- Maximum and Minimum sale
SELECT MAX(Sales_Amount) AS Max_Sale, MIN(Sales_Amount) AS Min_Sale
FROM Fact_Sales;
```

Expected Results:

Total_Sales

1700

Avg_Sale

850

Total_Transactions

Max_Sale	Min_Sale
1000	700

Q6 – GROUP BY & HAVING

```
-- Total sales per product
SELECT Product_ID, SUM(Sales_Amount) AS Total_Sales
FROM Fact_Sales
GROUP BY Product_ID;

-- Products with total sales > 1000
SELECT Product_ID, SUM(Sales_Amount) AS Total_Sales
FROM Fact_Sales
GROUP BY Product_ID
HAVING SUM(Sales_Amount) > 1000;
```

Expected Results:

Total sales per product:

Product_ID	Total_Sales
101	1000
102	700

Products with total sales > 1000:

(none in this dataset, because 1000 is not greater than 1000);